

Um algoritmo de *busca* (ou *exploração*) em um grafo é um algoritmo para percorrer os vértices de um grafo, a partir de um vértice inicial, até que seja atingida uma condição de parada. Dentre as condições de parada, destacam-se:

- percorrer todos os vértices do grafo,
- percorrer todos os vértices do grafo cuja distância ao vértice inicial seja menor que k ,
- atingir algum vértice específico do grafo,
- etc.

Dentre tantos detalhes, os algoritmos de buscas em grafo devem fazer uma "**marcação**" nos vértices para evitar que o algoritmo caia em um loop infinito (ao repetir sempre os mesmos vértices durante a busca).

Para evitar isso, uma estratégia comum é usar um sistema de cores nos vértices:

- um vértice BRANCO é aquele que ainda não foi atingido pela busca;
- um vértice CINZA é aquele que já foi atingido, mas, a partir dele, ainda pode ser usado para expandir a busca; e
- um vértice é PRETO se ele já foi atingido e todas as possibilidades de expandir a busca a partir dele já foram consideradas

1. Busca em Largura

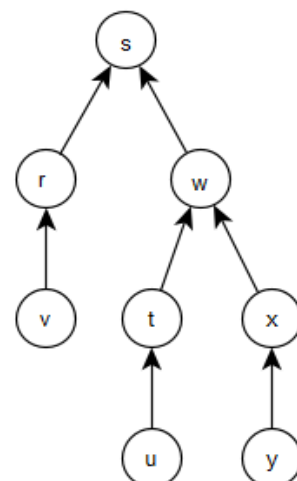
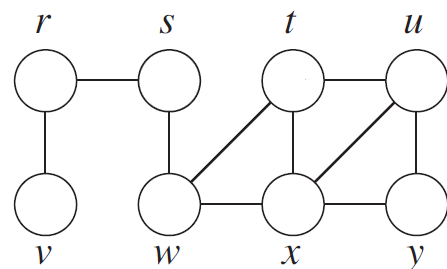
Algoritmo simples que expande os vértices na mesma ordem em que eles são atingidos. É útil para obter o **caminho mais curto** a partir do vértice inicial da busca. Algoritmo publicado no livro do "Cormen", 3a. edição, página 595:

O algoritmo BFS realiza uma busca em largura no grafo G usando o vértice s como vértice inicial da busca.

BFS(G, s)

```
1  for each vertex  $u \in G.V - \{s\}$ 
2     $u.color = WHITE$ 
3     $u.d = \infty$ 
4     $u.\pi = NIL$ 
5   $s.color = GRAY$ 
6   $s.d = 0$ 
7   $s.\pi = NIL$ 
8   $Q = \emptyset$ 
9  ENQUEUE( $Q, s$ )
10 while  $Q \neq \emptyset$ 
11    $u = DEQUEUE(Q)$ 
12   for each  $v \in G.Adj[u]$ 
13     if  $v.color == WHITE$ 
14        $v.color = GRAY$ 
15        $v.d = u.d + 1$ 
16        $v.\pi = u$ 
17       ENQUEUE( $Q, v$ )
18    $u.color = BLACK$ 
```

Exercício. Apresente a árvore e busca construída pelo algoritmo BFS a partir do vértice s . **Obs.:** na simulação do algoritmo, considere que, quando houver mais de uma opção de vértices a escolher, sempre será escolhido primeiro o vértice cuja letra ocorre antes na ordem alfabética.



Exemplo: Simulação da busca em profundidade, conforme livro do Cormen:

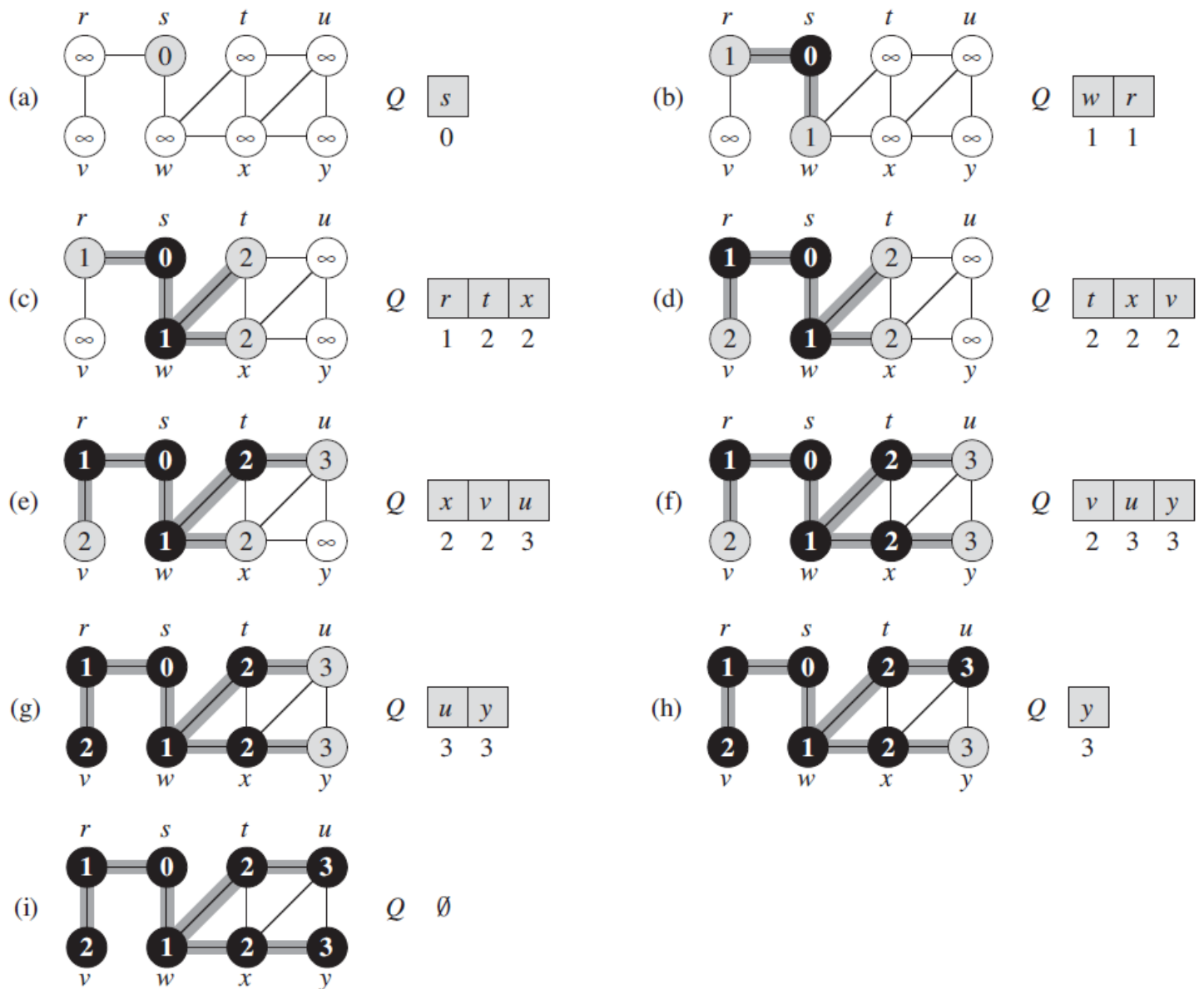


Figure 22.3 The operation of BFS on an undirected graph. Tree edges are shown shaded as they are produced by BFS. The value of $u.d$ appears within each vertex u . The queue Q is shown at the beginning of each iteration of the **while** loop of lines 10–18. Vertex distances appear below vertices in the queue.

Exercícios

1. Apresente a árvore e busca construída pelo algoritmo BFS a partir de cada um dos demais vértices do grafo.
2. Descreva como a busca em largura pode ser usada para avaliar se um grafo é conexo ou não.
3. Descreva como a busca em largura pode ser usada para contar quantas componentes conexas tem um grafo.

Um algoritmo de *busca* (ou *exploração*) em um grafo é um algoritmo para percorrer os vértices de um grafo, a partir de um vértice inicial, até que seja atingida uma condição de parada. Dentre as condições de parada, destacam-se:

- percorrer todos os vértices do grafo,
- percorrer todos os vértices do grafo cuja distância ao vértice inicial seja menor que k ,
- atingir algum vértice específico do grafo,
- etc.

Dentre tantos detalhes, os algoritmos de buscas em grafo devem fazer uma "**marcação**" nos vértices para evitar que o algoritmo caia em um loop infinito (ao repetir sempre os mesmos vértices durante a busca).

Para evitar isso, uma estratégia comum é usar um sistema de cores nos vértices:

- um vértice BRANCO é aquele que ainda não foi atingido pela busca;
- um vértice CINZA é aquele que já foi atingido, mas, a partir dele, ainda pode ser usado para expandir a busca; e
- um vértice é PRETO se ele já foi atingido e todas as possibilidades de expandir a busca a partir dele já foram consideradas

1. Busca em Profundidade

A busca em profundidade é um algoritmo que, como estratégia, expande primeiro o vértice que foi atingido por último no processo de busca.

Sua estratégia é aquela de um algoritmo de backtracking: avançamos na busca até não ser mais possível avançar; nessa situação, retrocedemos um passo e tentamos uma solução alternativa.

Versão simplificada do algoritmo

Apresentamos, inicialmente, uma versão simplificada do algoritmo para realizar uma busca em profundidade em um grafo G a partir de um vértice v :

DFS(G, v)

```
1 for each vertex  $u \in G.V$ 
2    $u.color = WHITE$ 
3    $u.\pi = NIL$ 
4 DFS-VISIT( $G, v$ )
```

DFS-VISIT(G, u)

```
1  $u.color = GRAY$ 
2 for each  $v \in G.Adj[u]$ 
3   if  $v.color == WHITE$ 
4      $v.\pi = u$ 
5     DFS-VISIT( $G, v$ )
6  $u.color = BLACK$ 
```

Versão completa do algoritmo

A versão completa usa um *contador de tempo* e dois atributos adicionais:

- *tempo de descoberta* de um vértice: tempo no qual o vértice se torna CINZA;
- *tempo de finalização* de um vértice: tempo no qual o vértice se torna PRETO.

Algoritmo publicado no livro do "Cormen", 3a. edição, página 604:

DFS(G)

```
1 for each vertex  $u \in G.V$ 
2    $u.color = WHITE$ 
3    $u.\pi = NIL$ 
4  $time = 0$ 
5 for each vertex  $u \in G.V$ 
6   if  $u.color == WHITE$ 
7     DFS-VISIT( $G, u$ )
```

DFS-VISIT(G, u)

```
1  $time = time + 1$ 
2  $u.d = time$ 
3  $u.color = GRAY$ 
4 for each  $v \in G.Adj[u]$ 
5   if  $v.color == WHITE$ 
6      $v.\pi = u$ 
7     DFS-VISIT( $G, v$ )
8  $u.color = BLACK$ 
9  $time = time + 1$ 
10  $u.f = time$ 
```

Exercício. Apresente a árvore e busca construída pelo algoritmo DFS a partir do vértice x . **Obs.:** na simulação do algoritmo, considere que, quando houver mais de uma opção de vértices a escolher, sempre será escolhido primeiro o vértice cuja letra ocorre antes na ordem alfabética.

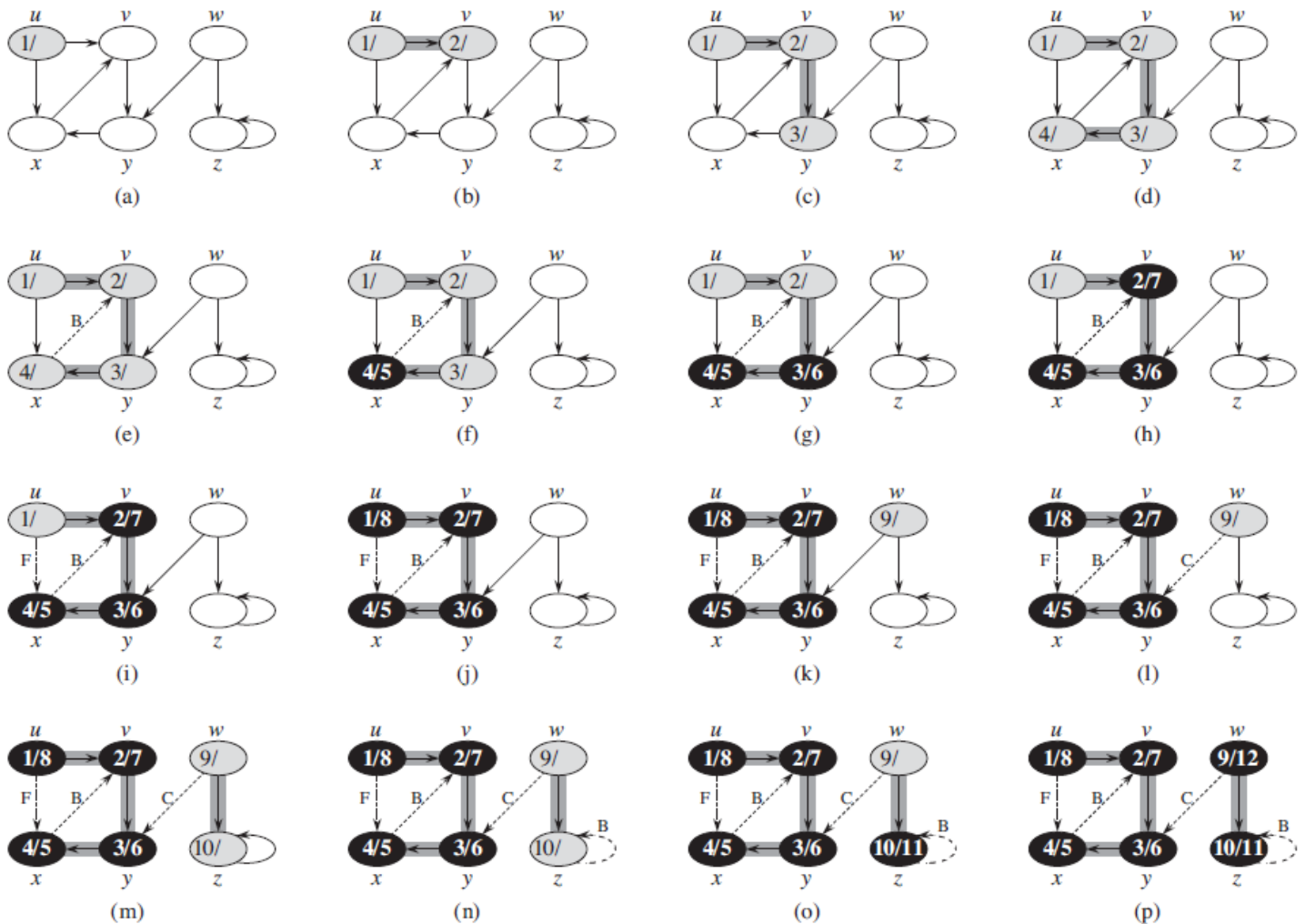
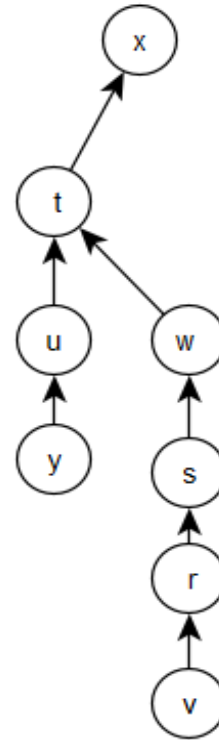
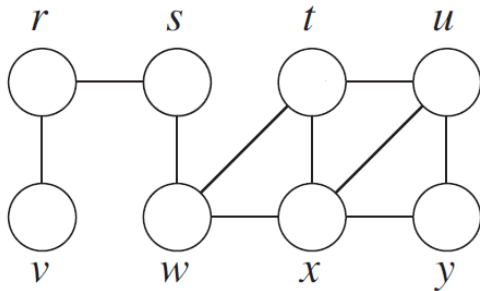


Figure 22.4 The progress of the depth-first-search algorithm DFS on a directed graph. As edges are explored by the algorithm, they are shown as either shaded (if they are tree edges) or dashed (otherwise). Nontree edges are labeled B, C, or F according to whether they are back, cross, or forward edges. Timestamps within vertices indicate discovery time/finishing times.